P1616R0
Using unconstrained template template parameters with constrained templates

Published Proposal, 2019-03-11

Authors:

Mike Spertus

Roland Bock

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Unconstrained template template parameters cannot accept constrained templates. This blocks important use cases and causes legacy templates to behave in unexpected ways as pointed out in Concepts TS issue 14. We propose addressing this.

Table of Contents

§ 1. The problem

It is our understanding that any template accepting an unconstrained template template parameter cannot accept a constrained template as pointed out in Concepts TS Issue 14. This creates problems with both legacy and new code.

To illustrate the problem, define the following classes

```
template<class T, class U>
struct Unconstrained { Unconstrained(U, V);};

template<class T, class U, class = enable_if_t<is_integral_v<U>>>
struct Sfinae { Sfinae(U, V); };

template<class T, class U> requires Integral<U>
struct Conceptized { Conceptized(U, V); };
```

In Concepts TS issue 14, the following was a blocker to adding concepts to sqlpp11.

```
// A template that takes a tuple and copies its arguments into another temp
template<typename Tuple, template<typename...> class Sink>
using copy_tuple_args = ...

copy_tuple_args<tuple<char *, int>, Unconstrained> u1; // OK. Unconstrainec
copy_tuple_args<tuple<char *, int>, Sfinae> s1;        // OK. Sfinae<char *
copy_tuple_args<tuple<char *, int>, Conceptized> c1;   // Ill-formed instea
```

As C++17 template template parameters are incompatible with concept-constrained templates, any attempt to concept constrain templates in existing code bases that make use of template template parameters will be likely to cause difficult or even impossible to fix breakage, discouraging the adoption of Concepts:

```
// Valid C++17 template
template<template<class, class> class TT, class A>
struct curry {
    template<class B>
    using type = TT<A, B>;
};

curry<Unconstrained, char *>::type<int> u3;  // OK. Unconstrained<char *, i
curry<Sfinae, char *>::type<int>  s3         // OK. Sfinae<char *, int>
curry<Conceptized, char *>::type<int> c3;    // Ill-formed instead of desir
```

There is not even a way to write new code with template template parameters without introducing restrictions on constraints, hindering use cases like Inferencing heap objects (P1069):

```
// Notation of P1069
auto u4 = make_unique<Unconstrained>(3, 5); // OK. unique_ptr<Unconstrainec
auto s4 = make_unique<Sfinae>(3, 5);        // OK. unique_ptr<Unconstrainec
auto c4 = make_unique<Conceptized>(3, 5);   // Ill-formed instead of desire
```

Likewise, ranges::to (P1206) awkwardly relies on standard library containers being unconstrained:

```
// copy a list to a vector of the same type, deducing value_type
Same<std::vector<int>> auto c = ranges::to<std::vector>(l); // OK, vector i
```

§ 2. The solution

We propose simply that an unconstrained template template parameter match concept-constrained classes. E.g.,

```
copy_tuple_args<tuple<char *, int>, Conceptized> c1;   // Proposed: Concept
curry<Conceptized, char *>::type<int> c3;              // Proposed: Concept
auto c4 = make_unique<Conceptized>(3, 5);              // Proposed: Concept
```

Note that violations of the constraints will still produce concept-aware error messages

```
// Proposed: Ill-formed with concept violation as desired
copy_tuple_args<tuple<int, char *>, Conceptized> c5;
```

§ 3. Without a language change, can we accept a general template as a template template parameter?

As far as we can tell, you can't. So use cases like the above are simply impossible. (Did we miss something? We'd love to be proven wrong).

§ 4. Why not create a new notation that matches any template?

While one could imagine some new notation for template template parameters that works with and without constraints, we believe that would still be an unsatisfactory solution for the following reasons

- All pre-C++20 template template parameters that were written in the past with the expectation that they would match any templates will fail to work with concept-constrained templates, discouraging existing code bases from conceptizing classes
- Teachers will have to teach the subtle point that any constrained class that might ever need to match a template template parameter should be SFINAE constrained rather than concept-constrained. We would much rather stop teaching SFINAE altogether instead of a new subtlety...
- A new notation could likely not be added before C++23. Our proposal could easily apply to C++20 as a wording fix to Concepts.

§ 5. What if we want to only match unconstrained templates?

We are unaware of use cases for the current behavior of only matching unconstrained concepts (especially compared to the significant use cases for this proposal). However, note that natural notations could easily be developed and added, e.g., by making the lack of constraints explicit:

```
template<template<class, class> requires {} class X> // Not proposed at thi
class OnlyMatchesUnconstrained {};

OnlyMatchesUnconstrained<Unconstrained> u5; // OK
OnlyMatchesUnconstrained<Sfinae> s5;        // OK
OnlyMatchesUnconstrained<Conceptized> s5;   // Ill-formed as desired
```